[CS3704] Intermediate Software Design and Engineering

Minhyuk Ko
Virginia Tech
10/29/2026

Announcements

- **HW3 due Friday (10/30) at 11:59pm**
  - **Reading Questions**
  - **Code Review Reflection + Activity**
  - **Code Contribution**
  - ***Please start as soon as possible!***
- **PM2.2 due next Friday (11/6) at 11:59pm**
  - **Progress Report**
  - **Black Box Test Plan**
- **Ut Prosim opportunities**



< This counts for
Ut Prosim points!

SPEND A WEEK IN EGYPT!

SOFTWARE ENGINEERING IN A GLOBAL CONTEXT



Complete the **CS 4094 Computer Science Capstone** with a **study abroad** component in **Egypt**! Study global and cultural factors affecting the Software Engineering industry alongside Egyptian students.

March 13-20, 2027

Alexandria, Cairo, and Alamein City

Apply by Nov. 1, 2026

(Priority Review: Oct. 1)







College of Engineering, Department of Computer Science

Scan the QR code or visit
https://bit.ly/CSVTinEgypt2027

Program Leader
Dr. Mohammed Seyam
seyam@vt.edu



< This does not count for Ut Prosim points!

Testing

Types of Testing
Test Cases
LLMs for Testing

Testing



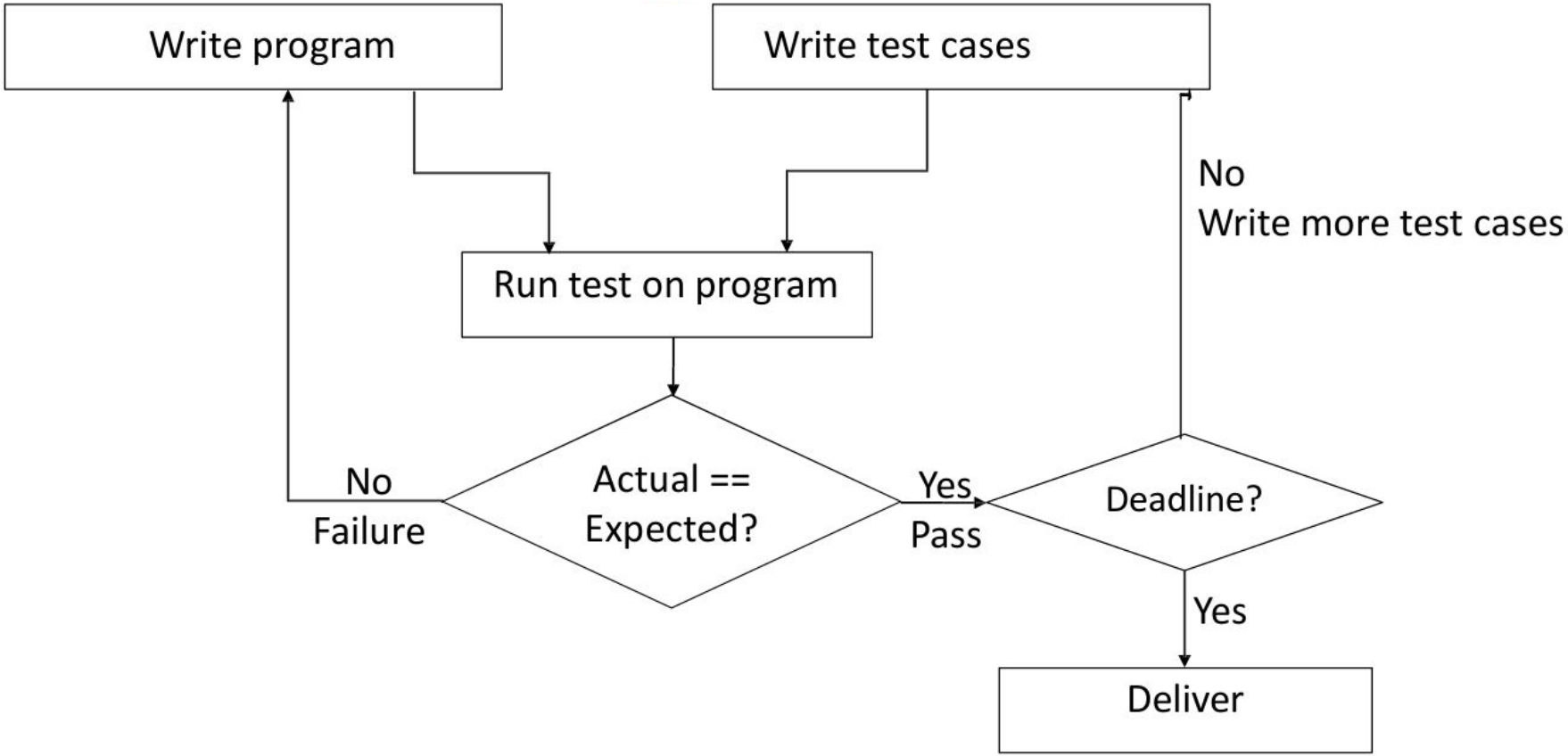
Goal: Execute software with intent of finding errors

- While you can't test until there is code to run, you can start planning testing when you're analyzing the requirements
- Includes Unit (Ut) and System (St) tests to verify code and functionality
- *Software Artifacts*: test code, bug database, test database, test inputs and outputs, documentation

What is Testing?

- Testing is the process of exercising a program with the specific intent of finding errors prior to delivery to the end user.
  - **Discovery**: *Executing a program or system with the intent of finding errors*. [Myers].
  - **Verify**: *Any activity aimed at evaluating an attribute or capability of a program or system and determining that it meets its required results*. [Hetzel]

Testing Process



Artemis II Outlook Failure



Testing Software is Hard

Hardware: If you are testing a bridge's ability to sustain weight and you test it with 1000 tons, you can infer that it will sustain 1000 tons

- *However, this kind of reasoning does not work for software systems*
  - software systems are not linear nor continuous
  - testing all possible input/output combinations is too expensive
  - the number of test cases increases exponentially with the number of input/output variables

Testing Software is Hard (cont.)

- Exhaustive testing

```
int max(int x, int y) {
     if (x > y)
          return x;
     else
          return y;
}
```

Number of possible test cases
(assuming 32 bit integers)
$-2^{32} * 2^{32} = \mathbf{2^{64}}$

- Random testing

Coverage is problematic

- i.e., If we pick test cases randomly it is unlikely that we will pick a case where x = y
- Non-uniform distribution
- When to stop?

Who s



devel

Understands t
but, will test
and, is driven

lly, both

When a Developer Finds a Bug vs When a Tester Finds a Bug

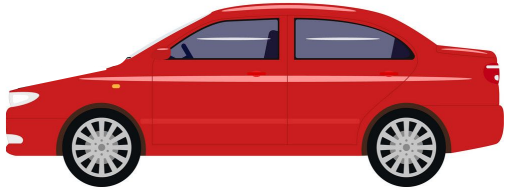
*Bug

Let's hide

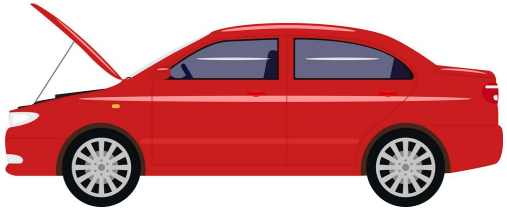
ProgrammerHumor.io

Types of Testing

1. **Black Box Testing:** ignores the internals of the program and focuses on functionality.



2. **White Box Testing:** uses code to guide testing.



Black Box Testing

- Focus on I/O behavior
  - If for any given input, we can predict the output, then the component passes the test.
  - Requires a test oracle
- Reduce test cases by using *equivalence classes* and *boundary value analysis*
  - ***Define a partition of the input domain***

Equivale

Minimize the nu
different equival

**Ex) Given a
odd, exampl
D_1 = {x is ev**

On one extreme
which turns into
whole input dom

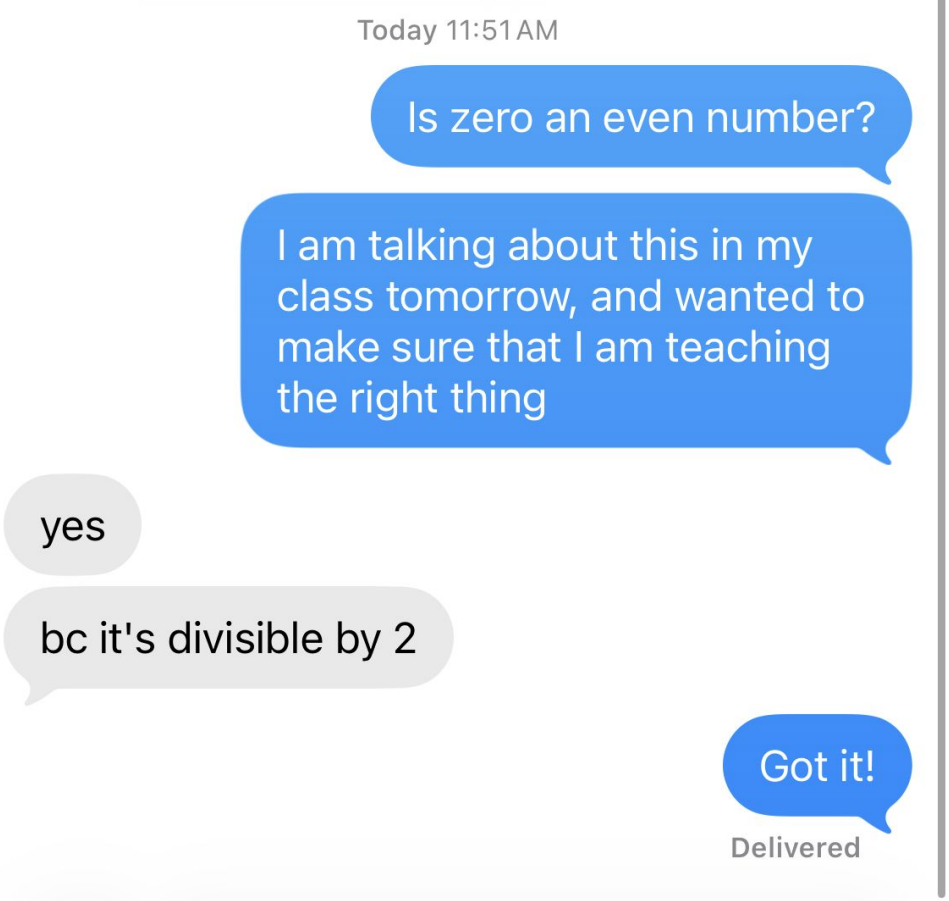
Ex) Test se

ntatives from

**is even or
ude:
D_4={x > 0}**

ne element
oosing a
one test case!

ses!



Boundary Value Analysis

- Similar concept to equivalence classes.
- Testing values around boundaries with conditional statements.

```
...
Scanner console = new Scanner(System.in);
System.out.print("Enter a month: ");
int num = console.nextInt();
if (num >= 1 && num <= 12) {
...
```

Ex) possible test set: {0, 1, 2, 11, 12, 13}

White Box Testing

- Use code to motivate testing
- Most popular: *unit testing*
  - **Unit:** individual functions or methods
- **Assert Statements:** Used to programmatically verify the *expected* output matches the *actual* output
  - Assertion libraries differ based on testing framework, programming language, etc.

Assert Statements Example

```
public static String evenOdd(int num) {
    if (num == 0 || num == 2 || num == 4 || num == 6 || num == 8) {
        return "Even";
    } else {
        return "Odd";
    }
}
```

Given the code above, will the following (JUnit) test cases pass or fail?

1. assertEquals("The expected output is Even", "Even", evenOdd(6)); // PASS
2. assertEquals("The expected output is Odd", "Odd", evenOdd(1)); // PASS
3. assertEquals("The expected output is Even", "Even", evenOdd(10)); // FAIL
4. assertTrue("The expected output is Odd", "Odd".equals(evenOdd(99)); // PASS
5. assertFalse("The expected output is Even", evenOdd(8).equals("Even")); // FAIL

White Box Testing (cont.)

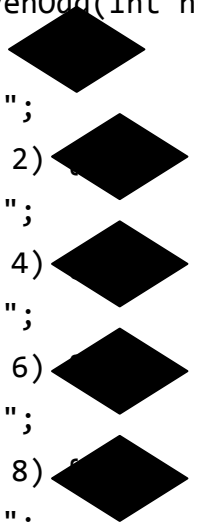
Measure for minimum number of unit tests…
Cyclomatic Complexity!

- Reminder: Number of independent paths in a program

  Cyclomatic Complexity = #decisions + 1

Review: Cyclomatic Complexity

```
public static String evenOdd(int num) {
      if (num == 0) {
            return "Even";
      } else if (num == 2) {
            return "Even";
      } else if (num == 4) {
            return "Even";
      } else if (num == 6) {
            return "Even";
      } else if (num == 8) {
            return "Even";
      } else {
            return "Odd";
      }
}
```



+1 = 6

Testing Metrics

The best case would be to execute all paths through the code, but there are some problems with this:

- the number of paths increases fast with the number of branches in the program
- the number of executions of a loop may depend on the input variables and hence may not be possible to determine
- most of the paths can be infeasible

Coverage Metrics

- ***Statement coverage:*** all statements in the programs should be executed at least once
- ***Branch coverage:*** all branches in the program should be executed at least once
- ***Path coverage:*** all execution paths in the program should be executed at least once

Example

```
areTheyPositive(int x, int y)
{
  if (x >= 0)
     print("x is positive");
  else
    print("x is negative");
  if (y >= 0)
    print("y is positive");
  else
    print("y is negative");
}
```

The following test set will give us statement and branch coverage:

T_1={(x=12,y=5), (x= -1,y=35), (x=115,y=-13),(x=-91,y=-2)}

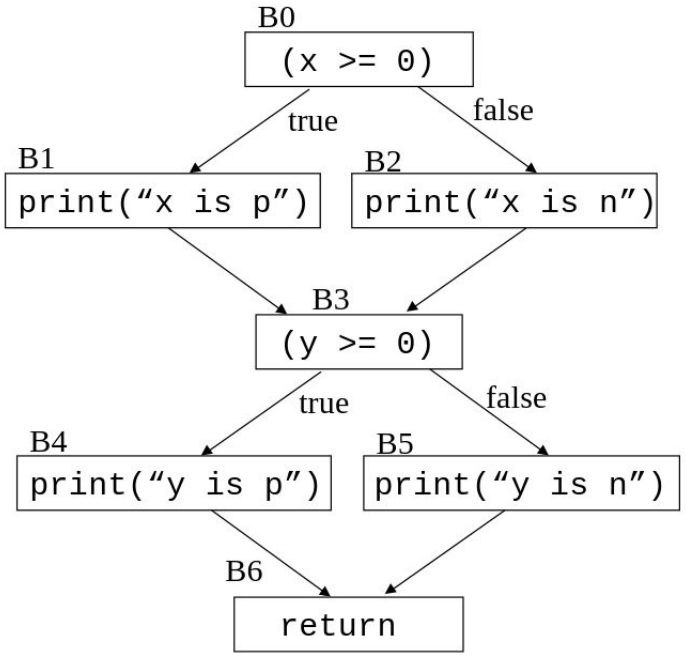
The following test set will give us statement and branch coverage with less test cases:

T_2 = {(x=12,y=-5), (x= -1,y=35)}

What about path coverage?

Example (cont.)

```
areTheyPositive(int x, int y)
{
  if (x >= 0)
     print("x is positive");
  else
    print("x is negative");
  if (y >= 0)
    print("y is positive");
  else
    print("y is negative");
}
```



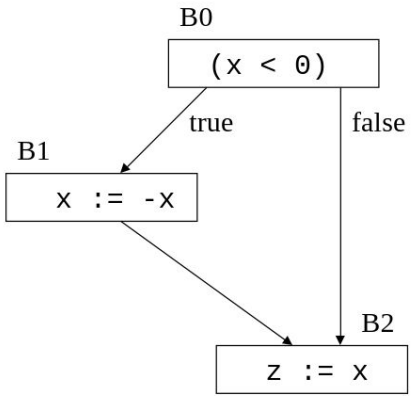
Path set:

[(B0,B1,B3,B4,B6);
(B0,B1,B3,B5,B6);
(B0,B2,B3,B4,B6);
(B0,B2,B3,B4,B6)]

T_2 {(x = 12, y = -5), (x = -1, y = 35)} only executes paths: (B0,B1,B3,B5,B6) and (B0,B2,B3,B4,B6). ***T_1 executes all paths, statements, and branches.***

Example 2

```
absoluteValue(int x)
{
  if (x < 0)
     x = -x
  return x;
}
```



T_1 = {x=-1} will give statement coverage.

What about branch/path coverage?

T_1 only executes one branch and path (B0, B1, B2). ***Path (B0, B2) is missing!***

How about T_2 = {(x=-1), (x=35)}?

This covers all paths, branches, and statements!

Moral: Minimizing test cases reduces time, but also potentially reduces coverage!

Writing Test Cases

- **Test case:** *Specific* functionality or feature to test
- **Required test case information for all test cases:**

1. Unique Identifier
2. Input
3. Expected Output
4. Actual Results

Black Box Test Cases

Test ID	Description	Expected Results	Actual Results
TestName (Test Author) Test Type	Preconditions: Steps: Test Inputs	Test Outputs	Actual Outputs

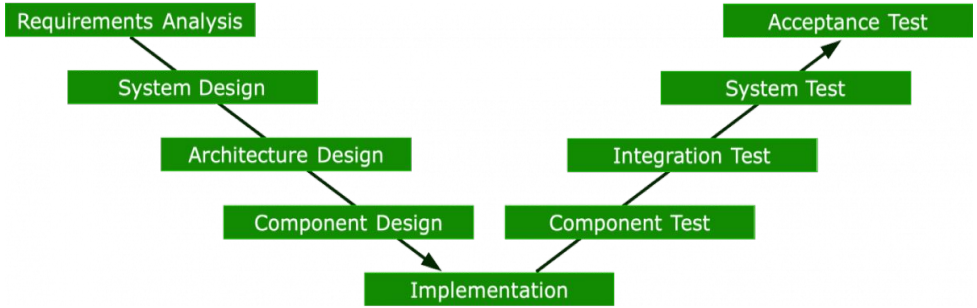
White Box Test Cases

```
    /**
     * Test evenOdd method with zero
     */
    @Test
    public void testEvenOddZero() {
        int num = 0;
        assertEquals("Expected output for 0 is Even", "Even", EvenOrOdd.evenOdd(num));
    }
```

Testing Strategies

Begin by "*testing-in-the-small*" and move toward "*testing-in-the-large*":

1. Unit Testing
2. Integration Testing
3. Validation Testing
4. System Testing



Strategy 1: Unit Testing

- **What are you testing:** Individual components on the smallest unit of software design (function, class, or subsystem)
- **Who?:** Carried out by developers
- **Goal:** Confirm that units are correctly coded and information flows in and out of modules properly.

Strategy 2: Integration Testing

- **What are you testing?:** Groups of subsystems, and eventually the entire system
- **Who?:** Carried out by developers (mostly, could also be testers)
- **Goal:** To construct tests to uncover errors associated with interfacing between units

Strategy 3: Validation Testing

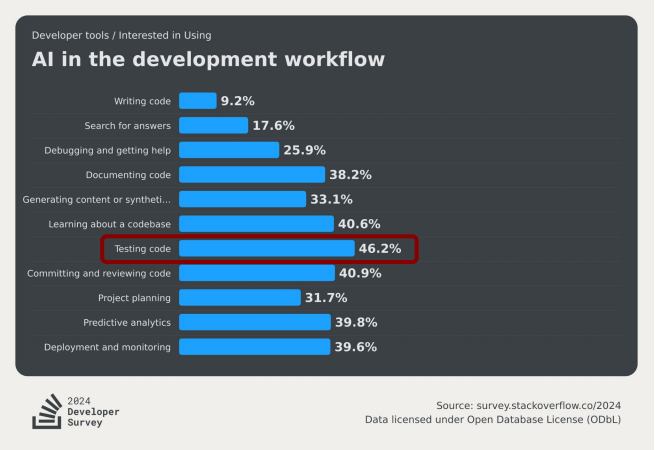
- Also known as *acceptance testing*
- **What are you testing?:** the system delivered by developers
- **Who?:** Carried out by testers and/or clients (through typical product interactions on a trial basis)
- **Goal:** Demonstrate that the system meets requirements and is ready to use

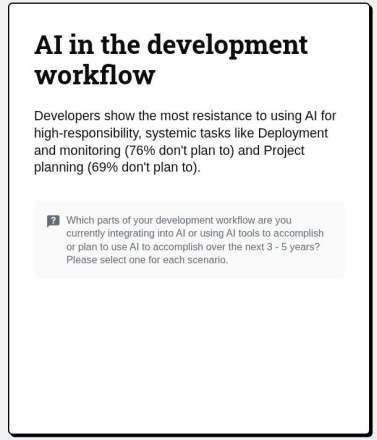
Strategy 4: System Testing

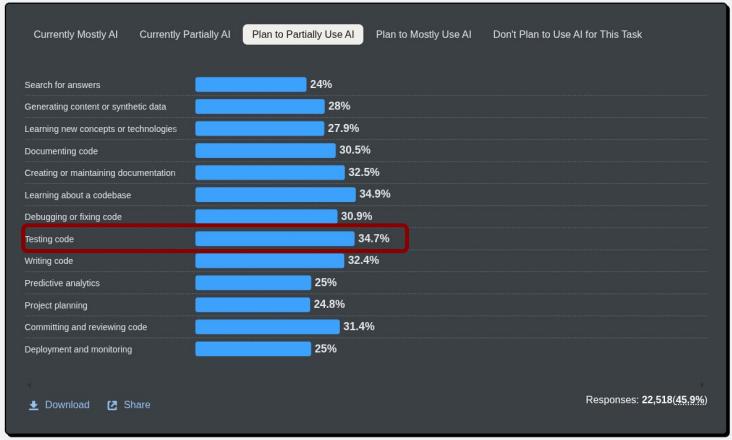
- **What are you testing?:** The entire system
- **Who?:** Developers and/or testers
- **Goal:** Determine if the system meets the requirements (functional and nonfunctional)
- **How is this different from validation testing?**
  - *Not* from the user's perspective

Automated Testing

Testing: Software engineers have long desired methods to automate software testing.







LLMs can accelerate automated testing with more context understanding…

AI for Testing: Benefits

LLMs can generate syntactically valid tests, often improving code coverage.

Table 1: Result on the overall coverage task. The results in parenthesis are the improvements over $cov@1$.

Model	Size	Correctness			Overall coverage		Line $cov@k$			Branch $cov@k$		
		syntax	execution	assertion	line	branch	$k=1$	$k=2$	$k=5$	$k=1$	$k=2$	$k=5$
GPT-3.5-turbo	N/A	**100**	97.43	40.40	96.27	93.65	88.35	90.02 (1.67)	92.14 (3.79)	81.87	84.32 (2.45)	87.55 (5.68)
GPT-4	N/A	**100**	92.33	54.16	94.94	92.81	85.65	87.77 (2.12)	90.04 (4.39)	78.89	81.93 (3.04)	85.39 (6.50)
GPT-4-turbo	N/A	**100**	94.79	56.24	96.08	94.81	85.46	87.87 (2.41)	90.81 (5.35)	78.62	82.06 (3.44)	86.64 (8.02)
GPT-4o	N/A	99.59	**98.30**	**52.99**	98.65	97.16	**90.23**	**92.16** (1.93)	**94.33** (4.10)	**84.05**	**86.89** (2.84)	**90.31** (6.26)
GPT-4o-mini	N/A	**100**	**99.92**	43.86	**98.76**	**97.58**	88.06	90.33 (2.27)	93.51 (5.45)	81.64	85.03 (3.39)	89.60 (7.96)
Gemini-1.0-pro	N/A	93.05	71.93	35.31	93.01	90.66	84.48	86.60 (2.12)	88.47 (3.99)	78.35	81.29 (2.94)	84.11 (5.76)
CodeLlama	7b	99.52	73.86	31.07	86.09	81.56	79.46	80.72 (1.26)	82.04 (2.58)	72.28	73.96 (1.68)	75.90 (3.62)
	13b	67.55	50.40	25.28	85.66	80.55	80.49	82.26 (1.77)	83.44 (2.95)	73.21	75.54 (2.33)	77.13 (3.92)
	34b	66.33	46.86	40.32	87.96	83.74	78.83	81.25 (2.42)	83.71 (4.88)	71.37	74.50 (3.13)	77.80 (6.43)
Llama3	8b	99.25	82.24	44.61	90.98	89.02	77.40	80.08 (2.68)	84.42 (7.02)	69.47	73.37 (3.90)	79.22 (9.75)
Llama3.1	8b	98.69	94.69	50.00	88.94	85.79	74.42	77.49 (3.07)	82.07 (7.65)	65.65	69.92 (4.27)	76.16 (10.51)
Gemma	7b	98.98	64.64	35.30	93.16	91.46	76.23	80.54 (4.31)	85.90 (9.67)	67.15	72.94 (5.79)	80.29 (13.14)
Starcoder-2-Instruct	15b	97.07	94.07	54.11	89.84	84.41	88.03	88.22 (0.19)	88.50 (0.47)	81.80	82.09 (0.29)	82.50 (0.70)
DeepSeek-coder	1.3b	96.05	82.48	38.66	81.22	75.99	75.89	76.50 (0.61)	77.09 (1.20)	69.06	69.90 (0.84)	70.70 (1.64)
	6.7b	97.42	82.43	40.43	93.48	91.61	82.40	84.74 (2.34)	87.97 (5.57)	75.29	78.73 (3.44)	83.46 (8.17)
	33b	99.21	83.57	50.75	94.86	91.92	85.47	87.38 (1.91)	90.30 (4.83)	78.49	81.23 (2.74)	85.12 (6.63)
CodeQwen	7b	**100**	84.26	46.36	90.73	86.90	84.53	85.33 (0.80)	86.71 (2.18)	77.66	78.94 (1.28)	80.95 (3.29)

[Wang]

AI for Testing: Case Study

Meta

Automated Unit Test Improvement using Large Language Models at Meta [Alshahwan]

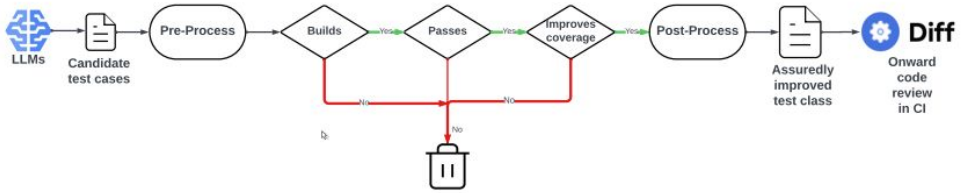


Figure 1: TestGen-LLM top level architecture (an instance of Assured Offline LLMSE [6]).

rank	test author	No, of tests	lines covered	diffs
1.	Threads Engineer	40	1,047	8
2.	Home Engineer	34	650	6
3.	Business Engineer	34	443	3
4.	Sharing Engineer	33	816	8
5.	Messaging Engineer	18	157	2
6.	**TestGen-LLM**	17	1,460	17
7.	Friends Engineer	12	143	2
8.	Home Engineer	10	273	2
9.	Creators Engineer	10	198	3
10.	Friends Engineer	10	196	5

“TestGenLLM verifies that its generated test classes successfully clear a set of filters that assure measurable improvement over the original test suite, thereby eliminating problems due to LLM hallucination…In an evaluation on Reels and Stories products for Instagram, 75% of TestGen-LLM’s test cases built correctly [and] 57% passed reliably…During Meta’s Instagram and Facebook test-a-thons, it improved 11.5% of all classes to which it was applied, with 73% of its recommendations being accepted for production deployment by Meta software engineers.

AI for Testing: Challenges

However, LLM-generated tests can be:

- Irrelevant
  - Testing useless or redundant test cases for the sake of improving code coverage
- Difficult to benchmark
  - LLMs perform significantly worse on large codebases with real-world code, overfit to metrics
- Unit-testing-centric
- Inconsistent
  - flaky
- Expensive
- Prioritize Java/Python
- …

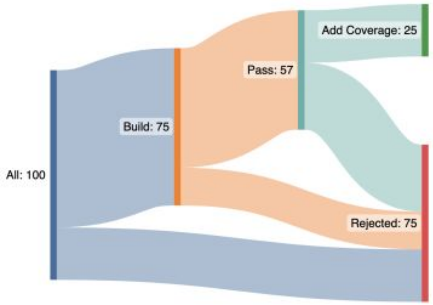


Table 9. Test Result Across Model

Model	Exception Failure	Compilation Failure	Assertion Failure	Passed
DeepSeek-R1-Distill-Qwen-32B	4	150	5	5
GLM-4.7-Flash	2	80	5	12
Llama-3.3-70B-Instruct	2	166	14	16
Qwen3.5-35B-A3B	5	148	4	11
gemini-2.5-pro	2	159	25	22
gpt-5-2025-08-07	4	102	16	37
gpt-oss-120b	12	137	12	8
us.anthropic.claude-opus-4-6-v1	2	117	16	27

Class Activity

Complete the following with your Project 2 group:

1. **Identify three test cases for your Project 2 application.**
2. **Write a black box test plan describing *at least three* test cases to for your system using the template in the slides (you need ten for PM2.2):**

Test ID	Description	Expected Results	Actual Results
TestName (Test Author) Test Type *Can ignore test type*	Preconditions: Steps: Test Inputs	Test Outputs	Actual Outputs *This column should be blank, unless feature is implemented!*

3. **Submit the test plan for attendance on Canvas [you may work together and submit once per group, but each person in attendance should be named the author of at least one test case].**

Due: before the end of the day (submit whatever you complete)

References

- Nadia Alshahwan, et al. “Automated Unit Test Improvement using Large Language Models at Meta”. Arxiv https://arxiv.org/pdf/2402.09171
- Tevfik Bultan. “272: Software Engineering; Lecture 10: Testing; Automated Testing”. 2008.
- Wenhan Wang, et al. “TESTEVAL: Benchmarking Large Language Models for Test Case Generation”. Association for Computational Linguistics, NAACL 2025. https://aclanthology.org/2025.findings-naacl.197.pdf
- RS Pressman. *“Software engineering: a practitioner's approach”.*
- Ruchika Malhotra and Anuradha Chug, *“Software Maintainability: Systematic Literature Review and Current Trends”*, 2016.
- Code Reviews lecture, University of Washington.